
Lightchain P2P

Pre-BETA Security, Functionality and Release Audit

Five-specialist audit of the lightchain-p2p workspace
Blockchain architecture · Blockchain engineering · AI architecture · AI
engineering · Release / DevOps

20 August 2026
Audited tree at commit ca3227c · read-only, file:line evidence throughout

CONFIDENTIAL - internal engineering document

Contents

1. Executive summary	3
2. Scope and methodology	4
3. Blockchain architecture	5
3.1 Ranked architectural risks	5
4. Blockchain engineering (code-level)	7
5. AI inference architecture	8
5.1 Ranked architectural risks	8
6. AI engineering (code-level)	10
6.1 Root cause: the seed-restore / worker-reset symptom	10
7. Release engineering and DevOps readiness	11
8. Consolidated BETA completion plan	12
8.1 P0 - money safety and correctness (blockers)	12
8.2 P1 - robustness (BETA runway)	12
8.3 Trust disclosure (one afternoon)	13
8.4 Release track (start on day one)	13
8.5 Sequencing	13
8.6 Test backfill before tagging	14
9. Accepted tradeoffs and out-of-scope	14
9.1 Accepted tradeoffs to state in the release notes	14
9.2 Deliberately out of scope for BETA	14

1. Executive summary

This report consolidates a five-specialist audit of the **lightchain-p2p** workspace performed on 20 August 2026: a blockchain architect, a blockchain engineer, an AI architect, an AI engineer, and a senior DevOps/release engineer. Each specialist read the actual implementation - the Electron application under `dev/lightchain-p2p/apps/chat`, the shared packages (`chain`, `wallet`, `inference`, `worker`), the Lightchain protocol mirror under `lightchain/` (48 repositories including the deployed contracts), and the verified source audit at `docs/proposals/AUDIT.md`. Every claim in this report carries file:line evidence against commit **ca3227c**.

Verdict. The engineering core is release-grade: key handling, the signing boundary, SIWE challenge vetting, per-frame answer-signature verification (which exceeds both the reference web client and the relay itself), worker-onboarding preflight, and the CI guard wall are all sound. What blocks a BETA release falls into three buckets:

- **Money-safety defects.** Eight blocker-class issues, headed by a confirmation threshold that is not chain-aware (a 50 ETH send or swap currently receives no confirmation dialog at all), a session-wedging bug that can crash the data-plane worker, spending limits that are silently not enforced on the main spend path, and a worker keystore password stored in plaintext.
- **The recovery half of the inference protocol is unwired.** Timeout refunds, quality disputes, delegate revocation and dispute-evidence persistence all exist on-chain or in encoders but are unreachable from the client - today any failure mode resolves to "you paid, sorry."
- **The distribution layer has never executed.** No production update channel (the committed upgrade link is a dev link), no signing certificates, no installer has ever been built, no crash reporting. Certificate procurement has multi-week external lead time and must start immediately.

Dimension	Status	Headline finding
Key handling and signing boundary	Sound	Sandboxed renderer never sees a key; EIP-1559 only; chainId pinned and re-checked (chain/src/send.ts:237-241)
SIWE authentication	Sound	Domain, URI, address, chain and expiry enforced before signing (inference/src/siwe.ts:128-189)
Answer verification	Sound - best in ecosystem	Per-frame worker signatures verified before decryption (inference/src/verify.ts:52-70)
Confirmation guard	Broken for ETH-denominated flows	Threshold compares ETH wei against a 100-token limit (workers/guard.mjs:44)
Inference failure recovery	Missing	claimTimeout, claimRefund, disputeJob, delegate revocation all unwired
Worker secrets at rest	Broken	Keystore password in plaintext settings.json and Docker env (handlers/worker.mjs:162-164)
Build / test / CI	Strong	970+ vitest tests green across 18 packages; ~10 bespoke CI guards
Release / update channel	Never executed	Dev upgrade link, placeholder multisig, no installer, no crash reporting

Table 1. Audit headline by dimension.

Reported symptom root-caused. The "restored my seed and the Earn worker setup reset" report was confirmed as an error-conflation bug, not data loss: the stake probe swallows every failure (including "ambiguous keystores") into a panel state pixel-identical to a fresh install. Nothing is deleted on restore; the real error simply never reaches the screen (Section 6).

2. Scope and methodology

The audit was strictly read-only. Scope covered the full master folder: the application monorepo `dev/lightchain-p2p`, the Lightchain protocol mirror `lightchain/` (`lightchain-contracts`, `lightchain-relay`, `lightchain-worker`, `lightchain-dispatcher`, `lcai-docs` and 43 sibling repositories), the verified protocol source audit `docs/proposals/AUDIT.md` (15 August 2026), and the Holepunch/Pear stack mirror the app is built on.

Contract interactions were checked field-by-field against the deployed Solidity source: the 18-word `getJob` decode, the `JOB_STATE` enum order, the answer-signature preimage (`keccak256(abi.encode(chainid, registry, jobId, sessionId, ciphertext)) + EIP-191`), and the session calldata all match `JobRegistry.sol` exactly. Test suites were executed where cheap; results below were reproduced this session.

Suite	Result	Suite	Result
apps/chat (vitest)	106 / 106 pass	packages/chain	209 / 209 pass
packages/inference	87 / 87 pass	packages/wallet	137 pass, 2 skipped
packages/worker	36 / 36 pass	13 further packages	all pass (prices, protocol, room, ui, preflight, host, ...)

Table 2. Automated test status, verified by execution on 20 August 2026.

Note: the frequently quoted "swap 25/25" and "surfaces 43/43" figures are not vitest suites - they are Electron CDP harnesses (`apps/chat/scripts/swap-check.mjs`, `surfaces-check.mjs`) that drive a real window against live chains. They were not re-run in this audit and must be re-run manually, with output archived, before any release tag.

Correction to a standing assumption. The app does not need the hardcoded `AI_CONFIG_ADDRESS 0x2e832E...D402` from the public docs. That address is the `AIConfig` implementation; the application resolves both contract addresses live from the `WorkerRegistry` genesis predeploy (`packages/chain/src/lightchain.ts:97-107`) and the worker container consumes the published `proxy` (`packages/worker/src/network.ts:72-73`). This is the correct pattern for upgradeable UUPS contracts; the earlier "config load failed" class of bug is fixed and regression-tested (`packages/worker/src/worker.test.ts:163-167`).

3. Blockchain architecture

All keys, signing and RPC live in the Bare worker; the renderer is fully sandboxed (electron/main.js:290-292: sandbox:true, nodeIntegration:false, contextIsolation:true) and the preload exposes no chain surface. The renderer sends intents over newline-delimited JSON and never sees a private key. Table 3 maps every on-chain touchpoint with its verdict.

Touchpoint	Verdict	Evidence and notes
Vault / keystore	Sound	BIP-39 + optional 25th word, scrypt N=262144, m/44'/60'/0'/0/i, sealed 0600 (wallet/src/vault.ts:46,51)
Signing boundary	Sound	EIP-1559 only; node chain id re-checked before signing (chain/src/account.ts:184-191, send.ts:237-241)
SIWE sign-in	Sound	Challenge held to domain/URI/address/chainId/expiry; recovered signer compared before submission (siwe.ts:128-188)
Prepaid funding	Sound, one gap	Guarded depositAndAuthorize(delegate) (handlers/ai.mjs:500-534); no revocation path (risk A2)
Sessions (sortition + classic)	Sound	Both flows; session id parsed from SessionCreated log topic (inference/src/conversation.ts:200-279)
Job submission / EIP-4844 blobs	Sound, centralised	Encrypted prompt uploaded as blob; submitJobOnBehalf paid by delegate (conversation.ts:389-441)
Answer verification	Sound - strongest part	Every relay frame verified before decryption; digest matches JobRegistry.sol:683-686 (verify.ts:52-70)
Disputes (equivocation)	Sound	disputeResponseMismatch correctly gated (lightchain.ts:332-353; JobRegistry.sol:664-700)
Worker registration / 50,000 LCAI stake	Sound process, unguarded tx	Stake read live via getMinWorkerStake(); stake tx signed inside Docker, outside the guard (worker/src/commands.ts:90-105)
Bridge (Hyperlane warp route)	Sound client, fragile rail	Chain-verified addresses, exact approvals, mandatory disclosure; no message-id persistence (handlers/bridge.mjs)
Uniswap v3 swap	Sound	Canonical SwapRouter02/QuoterV2, plan re-derived at send, slippage from fresh quote (chain/src/uniswap.ts:53-68)
RPC resilience	Mostly sound	RpcPool with benching for six chains; but Lightchain signing path hangs off a single endpoint (risk A6)

Table 3. On-chain touchpoint map with verdicts.

3.1 Ranked architectural risks

- **A1 - Total dependence on the foundation control plane.** Sign-in, model list, sortition draw, blob upload, job submission and relay all route through chat-api.mainnet.lightchain.ai. If it is down or censors an address, a funded user can do nothing - although the contracts support client-submitted sessions and the code already encodes the calldata (lightchain.ts:267). Ship the direct-to-chain fallback.
- **A2 - Standing delegate allowance with no revocation.** Every ai.fund raises the delegate's allowance and nothing lowers it (ai.mjs:490-498); the delegate is a foundation server key. Add

ai.revokeDelegate (authorization off, allowance zero) and disclose at fund time. Single largest avoidable fund-loss surface.

- **A3 - The largest transaction bypasses the guard.** The 50,000 LCAI worker stake is signed by the Go binary inside the container - outside guard.allow, the ledger and history (worker/src/commands.ts:90-105). Pre-flight the exact stake and destination in the guard dialog; record the registration tx afterwards.
- **A4 - Worker keystore password in cleartext.** Written to plaintext settings.json and passed to Docker as an env var readable via docker inspect (handlers/worker.mjs:162-164; commands.ts:58-61). Seal under the wallet account via SealedStore.
- **A5 - Bridge has no recovery path.** No Hyperlane message id is recorded and no explorer indexes the deployment; a stalled transfer is "call the foundation" with no artifacts (handlers/bridge.mjs:306-308). Parse the DispatchId log, store it, and add a stalled-transfer runbook view.
- **A6 - Lightchain signing path uses a single RPC endpoint** (workers/main.mjs:600). Route reads through the existing two-endpoint pool (archive node already listed at chains.ts:78); keep broadcast single-shot as pool.ts mandates.
- **A7 - One-confirmation finality everywhere.** Settled ledger entries are never re-checked (handlers/wallet.mjs:437-452) on a chain that suffered a mainnet halt on 11 August 2026 (AUDIT section 5). Use confirmations 3+ for bridge/fund/stake-sized moves.
- **A8 (accepted trade) - The confirmation dialog is renderer-drawn.** A compromised renderer can self-confirm; thresholds and idle-lock remain worker-side and all failure modes refuse (workers/guard.mjs:13-27). Consider a main-process native dialog for above-threshold confirmations after BETA.

Trust model. Cross-checks against the deployed contracts and AUDIT.md found no mismatches: bytes32 content references, the 18-field job struct, and economic (not TEE) verification are all handled as the protocol actually implements them. The unmitigated centralisation - dispatcher, relay, disputer and blob submitter are one operator - is documented in code comments but never stated in the UI. A BETA handling real money needs that sentence in the app.

4. Blockchain engineering (code-level)

Code-level review of the keystore, vault, send path, SIWE, swap, bridge, prepaid flows and error mapping. Keystore V3 (script $N=2^{18}$, encrypt-then-MAC, constant-time compare), vault seed handling, and transaction construction are all solid. The defects found:

ID	Severity	Defect	Location
H1	High	Confirmation threshold not chain-aware: 100e18 wei compared in the spent chain's native unit, so native ETH sends and ETH-to-LCAI swaps of arbitrary size below 100 ETH get no confirmation. This dialog is the only human checkpoint since the password tier was removed (commit 327e443).	workers/guard.mjs:44,148; handlers/assets.mjs:651; handlers/swap.mjs:463
M1	Medium	Approve without zero-reset reverts on USDT-style tokens after a leftover allowance (USDT mainnet is a curated swappable input). User is stuck on a raw revert with no reset path.	swap.mjs:427; bridge.mjs:242; tokens.ts:61-67
M2	Medium	bridge.send never re-verifies the allowance it depends on; swap.send does. A window that approves X and sends Y produces a reverted transferRemote with gas burned.	handlers/bridge.mjs:259-297; cf. swap.mjs:458-460
M3	Medium	Worker keystore password persisted in plaintext settings.json (mode 0600, plain JSON); sole protection of the key securing the 50,000 LCAI stake.	handlers/worker.mjs:162-164 ; workers/main.mjs:450-454
M4	Medium	No persistent tracking for Ethereum-side transactions: swap/bridge never call recordTransaction; bridge arrival state lives only in renderer memory and is lost when the page closes - on a bridge with no explorer.	handlers/wallet.mjs:281-323; renderer/lib/bridge.js:198-221
L1	Low	Insufficient-funds errors unmapped on wallet.send, ai.fund, ai.withdraw; wallet.send has no balance pre-check at all, so users see raw node strings.	handlers/wallet.mjs:898-943; ai.mjs:500-574
L2	Low	ai.fund/ai.withdraw parse amounts with bare BigInt(): malformed input throws a raw SyntaxError; zero relies on a chain revert.	handlers/ai.mjs:505,547 (use the whole() helper, wallet.mjs:96-110)
L3-L6	Low	Deposit watcher ignores testnet; deliberate over-approval fallback in bridge quote decode; keystore script ceiling $n \leq 2^{22}$ is a generous DoS cap; 5 s requote drives 8-10 RPC calls per tick through public endpoints.	deposits.mjs:128-129; hyperlane.ts:129-135; keystore.ts:231; renderer/lib/swap.js:87-93

Table 4. Blockchain engineering defects, ranked.

Residual design risk (not a bug): local-data keys derive from `signMessage("Lightchain local data key v1: <purpose>")` (wallet/src/derived.ts:39-42) - a fixed public sentence, so a phishing signature of that string elsewhere would compromise room-registry and transcript keys. Protected in-app by SIWE allowlisting and the sandbox; document for BETA.

Untested critical flows: all worker-handler money logic (planSend math, planSwap slippage/approval interplay, routeFor quote handling, ledger reconcile); guard cross-chain denomination (H1) has no regression test; no sign-to-broadcast test through the handlers even against a mock node. The Electron harnesses exist but are manual and live-network only.

5. AI inference architecture

The inference pipeline as it actually exists in code: renderer sends intents over framed JSON IPC to the Bare worker; models are discovered from the consumer API but priced from chain (AIConfig.calculateJobFee, handlers/ai.mjs:31-38); sessions open via sortition (service creates the session) or classic flow (client sends createSession with the dispatcher's signature); prompts are folded with prior turns (6,000-char budget), encrypted to the ephemeral session key, uploaded as EIP-4844 blobs and submitted as jobs paid by the delegate from the prepaid balance; responses stream over the relay with per-frame signature verification before decryption; the on-chain commitment is compared post-answer and equivocation is disputable from the UI.

Pipeline stage	Verdict	Note
A. Discovery and pricing	Sound	Chain-sourced flat per-job fees; zero-worker or unpriced models disabled before a wasted draw
B. Auth and funding	Sound, custody caveat	Excellent SIWE vetting; standing delegate allowance is permanent spend authority
C. Session establishment	Sound	Both protocols handled; dispatcher assignment anchored on-chain (InvalidDispatcherSignature)
D. Submission and debit	Mechanically sound	Service is a custodial chokepoint; spending limits do not cover this path
E. Streaming and verification	Sound - best in ecosystem	Chunk dedup by seq; both wire dialects; verify-before-decrypt exceeds reference client and relay
F. Commitment and dispute	Partial	Only equivocation dispute wired, only for live single-frame answers
G. Timeout / garbage handling	Broken UX	Honest error text but no refund path; late answers silently dropped
Earn / worker lifecycle	Mostly sound	Genuinely good preflight and restart-loop detection; secrets at rest are the weak point

Table 5. Inference pipeline stages with verdicts.

5.1 Ranked architectural risks

- **I1 - No timeout-refund path despite one existing on-chain.** On timeout the client says "submitted and paid for" and stops (conversation.ts:424-430); claimTimeout slashes the worker and refunds the prepaid balance (JobRegistry.sol:601-649). Build a client-side job lifecycle tracker and a "Claim timeout refund" affordance.
- **I2 - Dispute evidence is memory-only.** Transcripts persist only the jobId; a restart inside the 1-hour dispute window forfeits the only implemented remedy. Persist ciphertext + signature per turn in the encrypted transcript log.
- **I3 - Spending limits silently do not apply to ai.ask.** limits.check/record are called only in room.ask (ai.mjs:852-875); "Spent today" undercounts exactly the flow it pretends to meter.
- **I4 - Foundation trust anchor never disclosed in the UI;** on the sortition flow the client never verifies the on-chain session matches what it sealed. Read getSession after openSession and compare worker + encWorkerKey.

- **I5 - Quality disputes (disputeJob, bond + similarity resolution) unreachable.** With TEE a stub this is the ecosystem's only quality lever (JobRegistry.sol:492-598) and the client does not expose it.
- **I6 - Worker keystore password plaintext** (same defect as A4/M3, seen from the AI side: it also encrypts the worker's ECDH key).
- **I7 - Session inactivity expiry unhandled** (sessionInactivityTimeout = 1800 s, AIConfig.sol:114): an ask after 30 idle minutes dies on a raw SessionNotActive revert. Catch it and reopen transparently.
- **I8 (accepted) - Renderer-drawn guard and broad pear:startWorker allowlist** - acceptable for BETA only if stated in release notes.

Structurally missing: a persistent job-lifecycle tracker; the disputeJob flow; model-weight distribution (nothing distributes weights anywhere in the stack - a Hypercore channel is the natural future fit); visible conversation semantics (prompt-folding with a 6,000-char budget over stateless jobs is invisible to users); session resume/reconnect; per-turn provenance UX.

6. AI engineering (code-level)

ID	Severity	Defect	Location
B1	High	Spending limits not wired to the Models page: limits.check/record called only in room.ask, never in ai.ask. User-set caps are decorative on the main spend path.	handlers/ai.mjs:644-707 vs 852-875
B2	High	One failed submit wedges the session permanently: #pending set, then putBlob/submit awaited outside the guard; every later ask throws "already waiting". The orphaned promise can later reject unhandled - a Bare worker crash ("worker exited").	inference/src/conversation.ts: 397-416, 186-194
B3	Medium	ai.start has no in-flight guard: two overlapping starts leak a Conversation socket and chain session, still billing-capable.	handlers/ai.mjs:602-603; conversation.ts:133-198
B4	Medium	workerPassword never validated until docker run fails; every Earn check can pass while registration is guaranteed to fail inside the container.	packages/worker/src/container.ts:82-94 (detection only)
B5	Medium	On testnet the worker can never start from the app: NETWORKS.testnet carries no contract addresses and the resolver named in the error is not exported.	worker/src/network.ts:75-87; commands.ts:107-113
B6	Low	Batch: ai.ask blocks zero balance but not $0 < \text{balance} < \text{fee}$; commitment badge matched by :last-child not jobId; renderer request() has no timeout; late frames after timeout keep filling evidence.	ai.mjs:670-672; models.js:902-904; ipc.js:40-50; conversation.ts:454-457

Table 6. AI engineering defects, ranked.

Response rendering was specifically hunted for injection: no markdown library, DOM nodes built via textContent only, links gated to http(s) through a main-process allowlist. **No XSS vector found.**

6.1 Root cause: the seed-restore / worker-reset symptom

Verified first: **no code path ties Earn readiness to the wallet.** wallet.import calls forgetInference() and re-keys the room registry - nothing else. The worker keystore (~/.lightchain-worker/keys), its password (settings.json) and on-chain registration are untouched. The "reset" is an artifact of how readiness is derived and how failures are conflated:

- **RC-1 (primary) - stakeProbe erases the difference between "no key", "ambiguous keys" and "chain unreadable".** Every exception is swallowed into {} or {unreachable:true} (handlers/worker.mjs:50-56,79-81); the renderer draws step 2 as "No key" - pixel-identical to a wiped install. A user who re-runs key setup after a restore and works around the writeKeystore refusal ends up with two keystores, i.e. ambiguity, i.e. a panel that reads "reset". Fix: return {problem: err.message} and render it.
- **RC-2 - one bad config field collapses the whole panel.** worker.status returns {configured:false} when resolveConfig throws for any reason, hiding an existing, funded, registered keystore behind "Not configured" (worker.mjs:255-256). If settings.json was regenerated by corrupt-write recovery, the password is gone even though nothing else is.
- **RC-3 - readiness is evaluated against the current network setting.** A flip back to mainnet for a testnet-registered worker reads as "Short 50,000 LCAI" - things that passed now fail, while

nothing changed on the right chain. Show which network the probe ran against.

Untested critical flows: the Conversation class has zero tests (the most stateful, money-adjacent class in the stack); all of handlers/ai.mjs and handlers/worker.mjs are untested; renderer DOM logic untested; no integration test of ai.ask against a mock relay socket despite the injection point existing for it.

7. Release engineering and DevOps readiness

Area	Status	Evidence
Build pipeline	Ready	tokens/icons/coins/markup generators each with --check mode enforced in CI (ci.yml:61-78); turbo orchestration
Packaging config	Ready	electron-forge 7.11: MSIX / DMG / AppImage (forge.config.js); Flatpak/Snap configured but undecided
Built artifacts	Partial	apps/chat/out package output exists (447 MB); no out/make - no installer has ever been produced
Reproducibility	Partial	Committed pnpm-lock.yaml, --frozen-lockfile in CI, packageManager pinned; no .nvmrc
Windows signing	Partial	signtool + RFC 3161 timestamp wired, Azure hook path ready; no real certificate exists
macOS sign / notarize	Partial	osxSign + osxNotarize wired and gated on env; no Apple Developer account - weeks of lead time (docs/signing-procurement.md)
OTA update path	Partial	pear-runtime updater wired end-to-end; but the committed upgrade link is a dev link (apps/chat/package.json:8-9) and pear.json multisig is all placeholders
Staged rollout	Missing	No canary channel, no GitHub Release, no pear stage/seed ever run, no seeders running
Versioning	Partial	All 0.1.0 + hardcoded BETA badge; no changelog; MSIX versioner breaks on prerelease tags
Rollback / apply	Partial	Documented stage-higher-version rollback; apply path has silent-hang failure modes (electron/main.js:345-386)
CI	Strong	Format/lint/typecheck/test/build plus ~10 bespoke guards; CDP harness wall currently continue-on-error
Crash reporting	Missing	No crashReporter, no Sentry, no log file ever written to disk - a broken BETA machine can offer you nothing
Config validation	Ready	resolveConfig fails fast; AI_CONFIG_ADDRESS regression-tested (worker.test.ts:163-167); preflight probes docker/GPU/disk
Dependency risk	Ready	tar pinned past a critical advisory with written justification; native-build allowlist; runtime-major guard

Table 7. Release-readiness state map.

Ranked gaps. (1) No production update channel - blocks OTA, the only update mechanism on MSIX/AppImage/DMG. (2) No signing certificates - procurement has multi-week external lead time; start now. (3) Nothing has ever actually been released. (4) OTA apply is fragile (silent hangs). (5)

Zero crash diagnostics. (6) Beta config pass incomplete; pear.json still names the template app. (7) No version identity. (8) Flatpak/Snap unresolved. (9) Process hygiene: 44 unpushed commits, placeholder CODEOWNERS, stale apps/chat README.

Bottom line for this track: roughly 2-3 days of in-repo work plus weeks of external procurement. The in-repo work is wave-3/4 material; the procurement clock must start on day one.

8. Consolidated BETA completion plan

Deduplicated across all five specialists (several defects were found independently by two or three, which is strong signal they are real). The full working document lives at `docs/BETA-PLAN.md` in the repository.

8.1 P0 - money safety and correctness (blockers)

#	Issue	Fix
P0-1	Confirmation threshold not chain-aware (H1): arbitrary-size ETH sends/swaps below 100 ETH get no confirmation.	Per-chain thresholds or force-confirm non-Lightchain native flows; regression test.
P0-2	Failed submit wedges session forever; orphaned promise can crash the worker (B2).	Clear #pending in a catch; no-op-catch the abandoned promise; backfill Conversation tests.
P0-3	Spending limits decorative on the main spend path (B1/I3).	Move feeFor + limits.check/record into ai.ask.
P0-4	Timed-out jobs are permanent losses (I1).	Job lifecycle tracker; "Claim timeout refund" via claimTimeout/claimRefund.
P0-5	Standing delegate allowance, no revocation (A2).	ai.revokeDelegate (authorization off + allowance 0); disclose at fund time.
P0-6	Worker keystore password plaintext (A4/M3); never validated until docker run fails (B4).	Seal under the wallet account; validate by local decrypt at adopt/doctor time.
P0-7	Earn readiness conflation - the reported "worker reset" symptom (RC-1/2/3).	Surface probe failure reasons; per-field tolerant config; show probed network.
P0-8	The 50,000 LCAI stake tx bypasses guard, ledger and history (A3).	Guard-dialog pre-flight with exact stake + registry; record registration tx.

Table 8. P0 blockers.

8.2 P1 - robustness (BETA runway)

(1) USDT-style approve zero-reset. (2) bridge.send allowance re-check. (3) Chain-aware ledger for swap/bridge txs; persist bridge pending state and the Hyperlane DispatchId. (4) Persist dispute evidence in the encrypted transcript log. (5) ai.start in-flight guard. (6) Testnet worker addresses or explicit "unsupported". (7) Two-endpoint RPC pool for Lightchain reads. (8) Confirmations 3+ for bridge/fund/stake moves. (9) Small correctness batch: whole() amount parsing, insufficient-funds mapping, $0 < \text{balance} < \text{fee}$ block, jobId-matched commitment badge, IPC request timeout, session-inactivity reopen. (10) Requote cost throttling. (11) disputeJob quality-dispute flow with bond UX.

8.3 Trust disclosure (one afternoon)

Nothing user-facing states that the dispatcher, relay, disputer and blob submitter are one foundation operator, or that wrong-but-plausible answers are not client-detectable (only equivocation is disputable). Add this to the funding dialog and an About/Security screen. The code comments already say it; BETA users deserve the same honesty.

8.4 Release track (start on day one)

#	Item	Detail
R-1	Production update channel	pear touch under a real multisig (3+ seeding machines); replace the dev upgrade link; fill pear.json pubkeys and namespace. Blocks all OTA.
R-2	Signing certificates	Longest lead - begin today. Windows OV or Azure Artifact Signing; Apple D-U-N-S + Developer enrolment. Make check-signing.mjs --require blocking.
R-3	Execute an actual release	Build out/make installers, create a GitHub Release, pear stage + seed, stand up 2+ always-on seeders, OTA smoke test on win32/darwin/linux.
R-4	Harden the OTA apply path	try/catch + failure reply + retryable latch; timeout/reject on the main-side promise. Fix before OTA is the only update path.
R-5	Crash diagnostics	crashReporter.start(); ring-buffer log file teeing worker stdout/stderr; in-app "export diagnostics" (never keys/transcripts).
R-6	Version identity	Real version + BETA badge (renderer/lib/main.js:41); MSIX prerelease fix (forge.config.js:170); start a CHANGELOG.
R-7	Linux channels	Ship AppImage only for BETA; drop or store-gate Flatpak/Snap (they cannot receive OTA).
R-8	Repo hygiene	Push the 44 local commits (or set up the second remote); real CODEOWNERS; refresh stale README; promote the harness CI job to blocking.

Table 9. Release-engineering items.

8.5 Sequencing

Wave	Window	Contents
Wave 1	Days 1-3	P0-1, P0-2, P0-3, P0-6, P0-7 + Conversation tests. In parallel: kick off R-2 certificate procurement and the R-1 multisig/key ceremony.
Wave 2	Days 4-6	P0-4, P0-5, P0-8, P1 items 1-5, trust disclosure (8.3).
Wave 3	Days 7-9	Remaining P1, test backfill, R-4, R-5, R-6.
Wave 4	When certificates land	R-3 full release execution + OTA smoke test, live-harness re-runs archived, tag v0.9.0-beta.1, ship.

Table 10. Suggested sequencing.

8.6 Test backfill before tagging

Conversation lifecycle (submit-failure, timeout, chunk dedup, unsigned-frame refusal, cancel); handlers ai.mjs and worker.mjs (limits incl. day rollover, stakeProbe branches, error mapping); planSend/planSwap/routeFor/ledger-reconcile with mocked pools; guard cross-chain threshold regression; then re-run the live Electron harnesses (swap-check, bridge-check, surfaces-check, send-check) against mainnet and archive the output.

9. Accepted tradeoffs and out-of-scope

9.1 Accepted tradeoffs to state in the release notes

- The confirmation guard dialog is renderer-drawn: a compromised renderer can self-confirm. Thresholds, idle-lock and all failure modes remain worker-side and refuse closed (workers/guard.mjs:13-27).
- The pear:startWorker allowlist is broad; combined with the above, a compromised renderer can respawn an arbitrary worker module.
- Local-data keys derive from a signature over a fixed public sentence; a phishing signature of that sentence elsewhere would compromise local transcript keys.

9.2 Deliberately out of scope for BETA

Model-weight distribution (nothing distributes weights anywhere in the Lightchain stack - a Hypercore/Hyperdrive channel is the natural future fit); session resume/reconnect across restarts; per-turn provenance UI; multi-conversation parallelism; Flatpak/Snap distribution.

End of report. The working plan with live checkboxes is docs/BETA-PLAN.md; this report and that plan were produced from read-only audits - no source files were modified by the auditors.